

Do Machines Struggle Where Humans Do? Obfuscation as a Probe of Human and Machine Code Comprehension

Jack Le[✉]

University of Texas at Dallas

Richardson, USA

jvl210002@utdallas.edu

Abstract

Code obfuscation targets human comprehension, but current evaluations primarily measure language model performance degradation instead of whether models fail in the same way as humans. A direct comparison across five obfuscation tiers reveals that comprehension difficulty is non-monotonic. Only reasoning-tuned models mirror human difficulty patterns. This alignment, rather than raw accuracy, distinguishes robust comprehension from brittle pattern matching.

CCS Concepts: • Software and its engineering → Software reverse engineering; • Human-centered computing → Empirical studies in HCI.

Keywords: code obfuscation, program comprehension, large language models, human–model alignment

ACM Reference Format:

Jack Le. 2026. Do Machines Struggle Where Humans Do? Obfuscation as a Probe of Human and Machine Code Comprehension. In *Companion Proceedings of the 2026 ACM SIGPLAN International Conference on Systems, Programming, Languages, and Applications: Software for Humanity (SPLASH Companion '26)*, October 4–9, 2026, Oakland, CA, USA. ACM, New York, NY, USA, 2 pages. <https://doi.org/10.1145/3837729.3839573>

1 Problem and Motivation

Obfuscation is almost always measured without people, on two untested assumptions: that difficulty is monotonic, and that what burdens a person burdens an automated analyst, now usually an LLM. Recent work asks only whether obfuscation degrades a model [2, 7], never whether a model’s difficulty tracks a human’s, nor where comprehension breaks. If models fail where humans do, obfuscated code benchmarks human-like reasoning; if not, strong scores mask brittle competence.



This work is licensed under a Creative Commons Attribution 4.0 International License.

SPLASH Companion '26, Oakland, CA, USA

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2910-2/2026/10

<https://doi.org/10.1145/3837729.3839573>

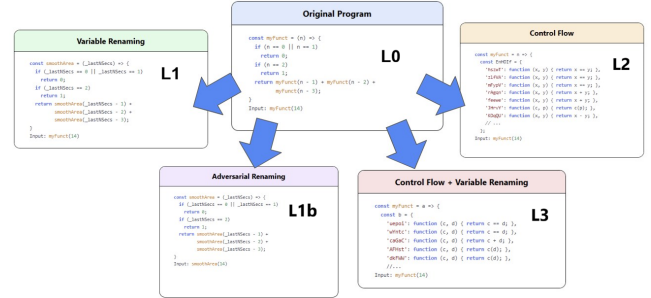


Figure 1. The five obfuscation tiers (L0, L1, L1b, L2, L3).

2 Approach

2.1 Obfuscation Scheme

Following Collberg et al. [3], I build five tiers (Figure 1), each verified execution-equivalent on the study’s inputs. **L0**: the original function with its name masked. **L1**: uninformative renaming; short, disjoint identifiers force structural reasoning. **L1b**: adversarial renaming; misleading names drawn from one semantic domain of the CodeSearchNet test split [5], a Stroop-like trap. **L2**: control-flow flattening; dispatchers and indirect jumps obscure execution order. **L3** combines both, an upper bound on difficulty. Tiers use ObfuXtreme (Python) [11] and javascript-obfuscator (JavaScript) [6], calibrated to what a supervised human session supports.

2.2 Dual-System Theory

Dual-process theory [4] models comprehension as fast System 1 and deliberate System 2. Obfuscation acts as a cognitive tax, stripping System-1 cues and forcing System-2 tracing. I test this via response time and chain-of-thought (CoT) length. Adversarial renaming (L1b) is the sharpest probe: like a Stroop task [12], a variable named `_lastNsecs` that tracks recursion depth baits a fast, confident, wrong answer.

2.3 Block Model

Schulte’s Block Model [10] formalizes comprehension as atoms (identifiers), blocks (methods), relations (data flow), and macro structure (purpose). Adversarial renaming corrupts atoms (identifier-perplexity spikes, semantic distance

[8]), flattening attacks blocks and relations (dispatcher-state and dispatch-call counts), and CoT length proxies macro effort.

3 Study Design

3.1 Task and Data

I use output prediction: given a function and input, state the exact output. This gives one executable ground truth. Base programs are deterministic (cyclomatic complexity 4–8 [9], <15 lines). Dataset A is the human-study set, 10 HumanEval-X [15] snippets per language, enabling direct human–model comparison. Dataset B is disjoint and model-only (50 snippets from HumanEval-X, CruxEval-X [14], LeetCode [13] $\times$ 5 tiers = 250), with a May-2025 cutoff.

3.2 Human Study

Fifty undergraduate CS majors, self-reporting per-language experience in three bands, sat a supervised 75-minute session of 12 questions. Each saw every base snippet at one tier, balanced across participants, input held constant. Exact-match scoring against execution yielded 600 responses with paired correctness and response time.

3.3 Model Study

Twelve LLMs (0.5B–8B) span reasoning-tuned (e.g., DeepSeek-R1, Phi-4-mini, SmolLM3-3B), instruction-tuned, and coder-tuned regimes on the same tiers; seven enter the human comparison on Dataset A, nine on Dataset B. Standardized prompts sweep reasoning depth, memory load, distractors, token budget, and scaffolding (28,750 runs). Confidence is length-normalized mean token log-probability [1], z-scored per model $\times$ condition; a high-confidence-incorrect (HCI) answer is a wrong prediction ≥ 1 SD above its group mean.

4 Results and Contributions

Difficulty is non-monotonic and language-specific. Human accuracy falls from L0 (40.5%) to L3 (31.1%): JavaScript declines steadily (43% to 16%), flattening hurting less than renaming, whereas Python rises under renaming (38% to 53%), L3 exceeding L2. *Effort matches the dual-process account:* response time jumps by about a minute once code is obfuscated, then plateaus; errors cluster at time extremes, and correct L1b answers are slower than incorrect ones.

Reasoning-tuned models align with humans on difficulty, not language. They correlate with human task-level difficulty (Spearman $\rho = 0.30$ – 0.47 , $p < 0.005$); coder- and instruction-tuned models do not ($\rho = 0.02$ – 0.10) regardless of size, so the most accurate model is not the most human-like. Alignment holds across all three experience bands (mean $\rho \approx 0.24$ – 0.30 ; no experience $\times$ alignment interaction in a mixed model), but no model reproduces the experts' gain under adversarial renaming or the Python renaming gain: alignment reflects shared difficulty, not identical mechanism. *Failures localize by*

Block Model level: CoT length correlates negatively with accuracy ($\rho = -0.52$); control-flow failure tracks dispatcher-state complexity (relational level); adversarial renaming acts non-additively (atom level): displacement or perplexity spikes alone barely move accuracy, but together collapse it to 21%.

Contributions. (1) The first per-task human–model difficulty correlation, localized via the Block Model. (2) Evidence against monotonic difficulty: a reproducible Python renaming effect. (3) Only reasoning-tuned models track human difficulty. (4) Language-specific behavioral reporting of obfuscation strength.

References

- [1] Youssef Abdelsalam, Norman Peitek, Anna-Maria Maurer, Mariya Toneva, and Sven Apel. 2025. How do Humans and LLMs Process Confusing Code? [doi:10.48550/arXiv.2508.18547](https://arxiv.org/abs/2508.18547)
- [2] Abhilekh Borah et al. 2026. Testing LLMs' Robustness Under Logical Obfuscation. In *Proceedings of the Conference of the European Chapter of the Association for Computational Linguistics (EACL)*.
- [3] Christian Collberg, Clark Thomborson, and Douglas Low. 1997. *A Taxonomy of Obfuscating Transformations*. Technical Report 148. University of Auckland.
- [4] Jonathan St. B. T. Evans and Keith E. Stanovich. 2013. Dual-Process Theories of Higher Cognition: Advancing the Debate. *Perspectives on Psychological Science* 8, 3 (2013), 223–241. [doi:10.1177/1745691612460685](https://doi.org/10.1177/1745691612460685)
- [5] Hamel Husain, Ho-Hsiang Wu, Tiferet Gazit, Miltiadis Allamanis, and Marc Brockschmidt. 2019. CodeSearchNet Challenge: Evaluating the State of Semantic Code Search. [doi:10.48550/arXiv.1909.09436](https://arxiv.org/abs/1909.09436)
- [6] javascript-obfuscator contributors. 2025. javascript-obfuscator (package version 4.1.1). <https://github.com/javascript-obfuscator/javascript-obfuscator>
- [7] Xiao Li et al. 2025. A Systematic Study of Code Obfuscation Against LLM-based Vulnerability Detection. [doi:10.48550/arXiv.2512.16538](https://arxiv.org/abs/2512.16538)
- [8] Ye Liu, Rui Meng, Shafiq Joty, Silvio Savarese, Caiming Xiong, Yingbo Zhou, and Semih Yavuz. 2024. CodeXEmbed: A Generalist Embedding Model Family for Multilingual and Multi-task Code Retrieval. [doi:10.48550/arXiv.2411.12644](https://arxiv.org/abs/2411.12644)
- [9] Thomas J. McCabe. 1976. A Complexity Measure. *IEEE Transactions on Software Engineering* SE-2, 4 (1976), 308–320. [doi:10.1109/TSE.1976.233837](https://doi.org/10.1109/TSE.1976.233837)
- [10] Carsten Schulte. 2008. Block Model: An Educational Model of Program Comprehension. In *Proceedings of the Fourth International Workshop on Computing Education Research (ICER)*. 149–160. [doi:10.1145/1404520.1404535](https://doi.org/10.1145/1404520.1404535)
- [11] spyboy productions. [n. d.]. ObfuXtreme. GitHub repository. <https://github.com/spyboy-productions/ObfuXtreme> Last accessed 2025.
- [12] J. Ridley Stroop. 1935. Studies of Interference in Serial Verbal Reactions. *Journal of Experimental Psychology* 18, 6 (1935), 643–662. [doi:10.1037/h0054651](https://doi.org/10.1037/h0054651)
- [13] Yunhui Xia et al. 2025. LeetCodeDataset: A Temporal Dataset for Robust Evaluation of Code LLMs. [doi:10.48550/arXiv.2504.14655](https://arxiv.org/abs/2504.14655)
- [14] Ruiyang Xu et al. 2024. CRUXEval-X: A Benchmark for Multilingual Code Reasoning, Understanding and Execution. [doi:10.48550/arXiv.2408.13001](https://arxiv.org/abs/2408.13001)
- [15] Qinkai Zheng et al. 2023. CodeGeeX: Multilingual Benchmarking on HumanEval-X. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*. 5673–5684. [doi:10.1145/3580305.3599790](https://doi.org/10.1145/3580305.3599790)

Received 2026-06-25; accepted 2026-07-25